

# Release 1.0.2 — Components shop + terms

</pages/components/> becomes a spare-parts shop with a WhatsApp checkout; new </pages/terminos/> and shared [js/tienda.jsx](#).

**VERSION**

1.0.2

**DATE**

2026-10-03

**STACK**React 18 UMD · Babel  
standalone · Tailwind CDN**STATUS**

Pre-launch (placeholders open)

## 1 Summary

The old UI-component library at </pages/components/> is replaced by a shop of 30 spare parts in six categories (Oficina, Estudiante, Dev, Redes, Gamer, Otros). Visitors filter, search, sort, add to a cart and send the order as a pre-filled WhatsApp message to [+34 602 55 76 85](#). Prices are stored and charged in EUR (VAT included) and shown for reference in USD, MXN, GBP and ILS using European Central Bank rates. A new terms page covers the seller, returns (30 days), legal withdrawal (14 days), the 3-year guarantee, data protection and a withdrawal form.

## 2 Files

| File                                                    | Role                                                                                                                                                                                                                                                                   |
|---------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">pages/components/index.html</a> (525 lines) | Shop page: <a href="#">CATEGORIES</a> , <a href="#">GRADES</a> , <a href="#">PRODUCTS</a> , <a href="#">ProductCard</a> , <a href="#">CartDrawer</a> , <a href="#">ShopPage</a> . All inline in a <a href="#">text/babel</a> script.                                   |
| <a href="#">pages/terminos/index.html</a> (221 lines)   | Terms page: 11 <a href="#">SECTIONS</a> , <a href="#">Fill</a> (highlights unfilled [...] values), <a href="#">REVIEW_DAYS = 7</a> .                                                                                                                                   |
| <a href="#">js/tienda.jsx</a> (103 lines, new)          | Shared by both shop pages: <a href="#">SELLER</a> , <a href="#">RETURN_ADDRESSES</a> , <a href="#">RETURN_DAYS</a> , <a href="#">TERMS_URL</a> , <a href="#">CURRENCIES</a> , <a href="#">store</a> , <a href="#">useCurrency()</a> , <a href="#">CurrencySelect</a> . |
| <a href="#">js/layout.jsx</a>                           | Unchanged, but required first: provides <a href="#">React</a> hooks, <a href="#">CONTACT_EMAIL</a> , <a href="#">PageHero</a> , <a href="#">renderPage()</a> .                                                                                                         |

### Load order (both shop pages)

```
react 18.2.0 UMD → react-dom 18.2.0 UMD → babel-standalone 7.23.5  
→ /js/layout.jsx → /js/tienda.jsx → inline <script type="text/babel"> ... renderPage(ShopPage)
```

[tienda.jsx](#) has no imports: it reads [useState](#), [useEffect](#) and [CONTACT\\_EMAIL](#) as globals declared by [layout.jsx](#). Loading it first, or alone, throws a [ReferenceError](#).

## 3 Data model

[pages/components/index.html](#) · line 41

```
{ id: 'of-kb-hp840', cat: 'oficina', name: 'Teclado español retroiluminado',  
  compat: 'HP EliteBook 840 G5 / G6', price: 34.90, grade: 'estandar' }
```

| Field                 | Meaning                                                                                                                                                                                                      |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">id</a>    | Stable key: cart storage, image file names (1.0.3). Prefix = category ( <a href="#">of-</a> , <a href="#">es-</a> , <a href="#">dv-</a> , <a href="#">rd-</a> , <a href="#">gm-</a> , <a href="#">ot-</a> ). |
| <a href="#">cat</a>   | One of the six <a href="#">CATEGORIES[]</a> . <a href="#">id</a> ; also the <a href="#">?cat=</a> URL value.                                                                                                 |
| <a href="#">price</a> | EUR, VAT included. The only price the customer is charged.                                                                                                                                                   |
| <a href="#">grade</a> | <a href="#">estandar</a> (compatible, tested) or <a href="#">premium</a> (OEM-grade). Drives the badge.                                                                                                      |

## 4 Features

### 4.1 Catalogue view

- Category** chips; the choice is written to [?cat=<id>](#) with [history.replaceState](#), so categories are linkable. Unknown values fall back to [todos](#).

- **Search** is a case-insensitive substring match on `name + compat`. No hit shows a “Pedir por WhatsApp” link with the query.
- **Sort**: featured (array order), price ascending/descending, name ( `localeCompare(..., 'es')` ).

## 4.2 Currencies — `useCurrency()`

```
const RATES_URL = 'https://api.frankfurter.dev/v1/latest?base=EUR&symbols=USD,MXN,GBP,ILS';
// returns { currency, setCurrency, format(eur, target?), ratesDate }
```

- Starts with fallback rates hard-coded in `CURRENCIES`, then replaces them with the live ECB rates when the fetch succeeds; any failure keeps the fallbacks silently.
- Non-EUR amounts are prefixed `€` and formatted with the ISO code ( `MXN 821.17` ) so pesos and dollars cannot be confused; the EUR charge is always shown beside them.
- The selected currency persists in `localStorage['lm-currency']`.

## 4.3 Cart and checkout — `CartDrawer`

- Cart is `{ [productId]: qty }`, quantity capped at 99, persisted to `localStorage['lm-cart']`. On load, ids no longer in `PRODUCTS` are dropped.
- `store.get/set` wrap `localStorage` in `try/catch`, so private mode or blocked storage only loses persistence.
- Checkout requires: at least one line, a name, delivery (*Envío en España* with a 5-digit postcode, or *Recogida en Madrid*) and the terms checkbox.
- The order is a text message: lines, EUR total (plus the reference currency), name, delivery and the terms URL, sent to `wa.me/${SELLER.whatsapp}`. Shipping costs are confirmed in the chat before payment.
- The drawer is a `role="dialog"` with `aria-modal`; focus moves to the close button and Esc closes it.

## 4.4 Terms page

Eleven anchored sections ( `#vendedor` ... `#formulario` ) linked from each product card, the cart and the footer. Values from `SELLER` and `RETURN_ADDRESSES` that still start with `[` are rendered in an amber `<mark>` so they are visible before launch.

## 5 Known issues and limits

**5.1 Launch blockers.** `SELLER.name`, `nif`, `address` and both `RETURN_ADDRESSES` are still `[...]` placeholders, and catalogue prices are not final. Spanish consumer law requires the seller's identity on the terms page before selling.

- **5.2 Catalogue in HTML.** `PRODUCTS` and `CATEGORIES` are code inside the page. Editing stock or prices means editing JSX; there is no stock field yet.
- **5.3 Client-side only.** Products are rendered by React in the browser, so the delivered HTML contains no product names or prices: weak for search engines and no `Product` structured data.
- **5.4 In-browser compilation.** Babel standalone transpiles ~1,000 lines of JSX on every page view, and the Tailwind Play CDN generates CSS at runtime. Both are documented as not for production.
- **5.5 Rates request on every visit.** No caching of the ECB rates (they change once per working day).
- **5.6 Orders are not recorded.** The only record of an order is the WhatsApp chat; prices in the message are whatever the client computed.
- **5.7 Postcode check** accepts any 5 digits (e.g. `00000`, or codes outside Spain's 01–52 provinces).

## 6 What 1.0.2 is missing for the 1.2.0 JavaScript version

| # | Gap               | What 1.2.0 needs                                                                                                                                                                                                                                                       | Priority |
|---|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| 1 | Build step        | <code>package.json</code> + a bundler (Vite or esbuild) that precompiles JSX and builds Tailwind with its CLI. Removes <code>babel-standalone</code> and <code>cdn.tailwindcss.com</code> from every page.                                                             | BLOCKER  |
| 2 | ES modules        | Replace implicit globals with <code>import / export</code> : <code>layout.js</code> exports hooks/components, <code>shop/currency.js</code> exports <code>useCurrency</code> , <code>shop/seller.js</code> exports <code>SELLER</code> . Script order stops mattering. | BLOCKER  |
| 3 | Catalogue as data | Read products and categories from <code>public/db/productos.json</code> and <code>categorias.json</code> (already present, see 1.0.3) instead of the inline arrays; map <code>cat→category_id</code> , <code>price→price_eur</code> .                                  | HIGH     |

|   |                        |                                                                                                                                                                                                                               |         |
|---|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| 4 | Pure, testable logic   | Extract cart reducer, order-message builder, price formatting and postcode validation into plain functions with unit tests.                                                                                                   | HIGH    |
| 5 | Pre-rendered HTML      | Generate the shop (and optionally one page per product using the <code>slug</code> field) at build time, with JSON-LD <code>Product</code> / <code>Offer</code> , plus <code>sitemap.xml</code> and <code>robots.txt</code> . | HIGH    |
| 6 | Deploy pipeline        | Once source $\neq$ output, GitHub Pages must serve a build: a GitHub Actions workflow that builds to <code>dist/</code> and deploys, keeping <code>CNAME</code> , <code>404.html</code> redirects and trailing-slash URLs.    | HIGH    |
| 7 | Rates caching          | Cache the ECB response with its date (or fetch at build time) and only refetch when stale.                                                                                                                                    | MED     |
| 8 | Stock and active flags | Honour <code>stock</code> and <code>active</code> from the JSON (hide or label out-of-stock parts).                                                                                                                           | MED     |
| 9 | Business data          | Seller identity, return addresses, final prices (not code, but they block launch in any version).                                                                                                                             | BLOCKER |